Winter 2021

493-0: VLSI Algorithmics

Lab 2. Catapult GUI, Interface Constraints and Design Analysis

Disclaimer: This tutorial was based on original ***Mentor Graphics*** tutorials. Slight modifications were required due to divergences in the versions of the tools and deprecated instructions. All credits go to the authors of the original tutorials.

In this lab you will learn how to:

- (1) Synthesize a design in Catapult using scripts and the Catapult GUI
- (2) Map an ac_channel variable to a hardware interface component
- (3) Analyze the design using the Catapult Gantt Chart and Catapult Design Analyzer to understand how the C++ has been synthesized to hardware
- (4) Run automatic verification of the C++ and the RTL
- (5) Enable pipeline flushing

Before we begin with the tutorial...

Remember to check Tutorial 0 to assert how to setup ssh to access the Wilkinson server and configure and run Catapult for the first time. The link is here.

Steps:

0. Access the Wilkinson server with your NetID and open a Terminal.

1. Load the cad tools environment by running command **(1)**:

```
>> source ~/envsetup-catapult.sh
```
(1)

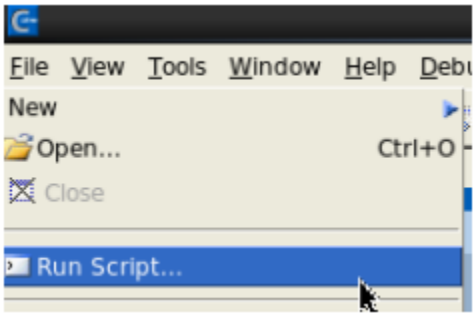
2. Navigate to the Labs directory and then to Lab2 using command **(2)**:

```
>> cd ~/Labs/Lab2
```
(2)

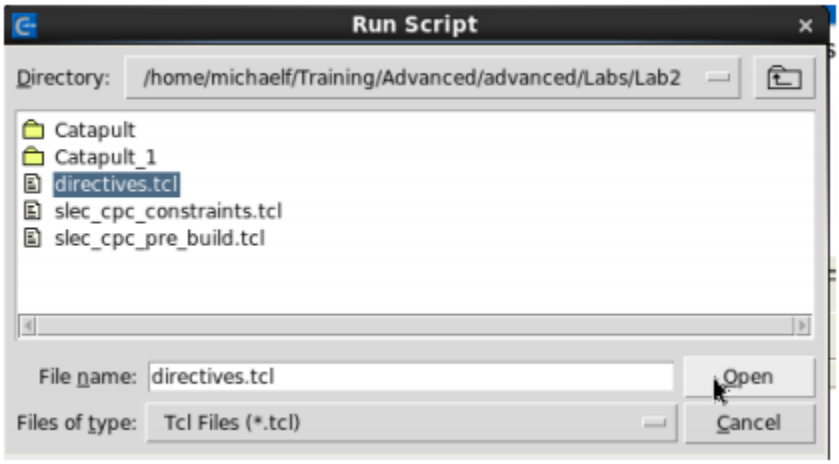
3. Let's launch Catapult now while being in this directory by running **(3)**:

```
>> catapult -product ultra
```
(3)

4. Go to the Catapult File menu and select "Run script"

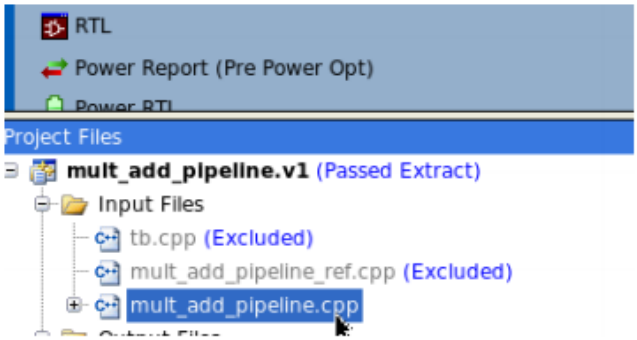


5. Select the directives.tcl file and click Open. This will add the input files, enable the verification flow, and synthesize the design.



6. There are 3 files for this design, tb.cpp and mult_add_pipeline_ref.cpp which are part of the testbench, and mult_add_pipeline.cpp which is the synthesizable design.

7. Double-click on the mult_add_pipeline.cpp file in the project files folder to open in the text editor.

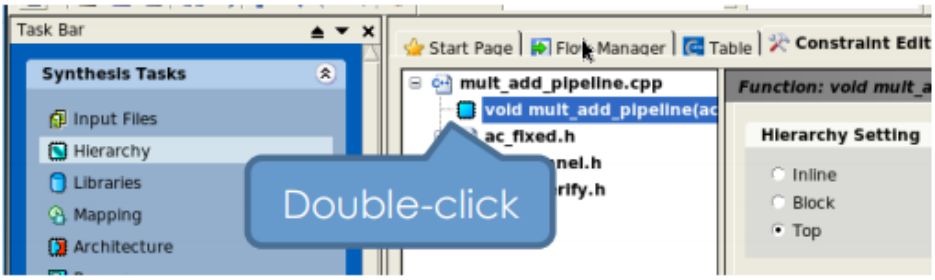


8. The design has ac_channel streaming interfaces for a,b, and c inputs and the result output. The design does a multiply-add of a, b, and c and multiplies by gain if gain_adjust is set to true.

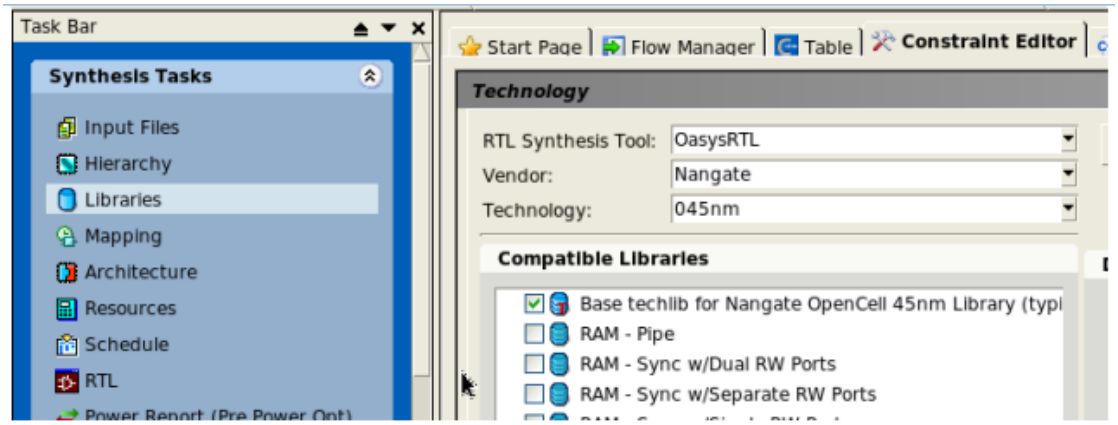
```
#pragma hls_design top
void CCS_BLOCK(mult_add_pipeline)(ac_channel<ac_int<11,false> > &a,
                                  ac_channel<ac_int<14,false> > &b,
                                  ac_channel<ac_int<25,false> > &c,
                                  ac_fixed<10,2,true> gain, bool gain_adjust,
                                  ac_channel<ac_int<30,false> > &result){
  ac_int<25,false> product = a.read() * b.read();
  ac_int<26,false> sum = product + c.read();

  if(gain_adjust)
    sum = ((ac_fixed<26,26,false>)(sum*gain)).to_uint();
  result.write((ac_int<30,false> )sum<< 4);//NOTE: cast sum to ac_int<30,false
}
```

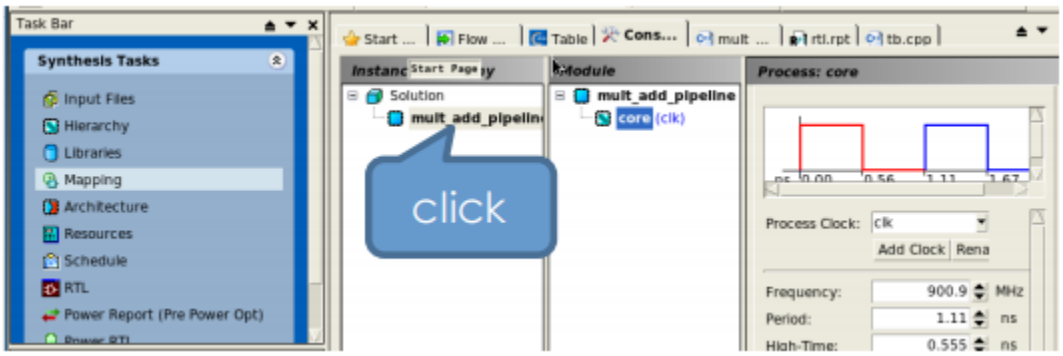
9. Click on the Hierarchy Button in the Task Bar. Note that mult_add_pipeline has been set to the toplevel design. Double-click on mult_add_pipeline to cross-probe to the C++.



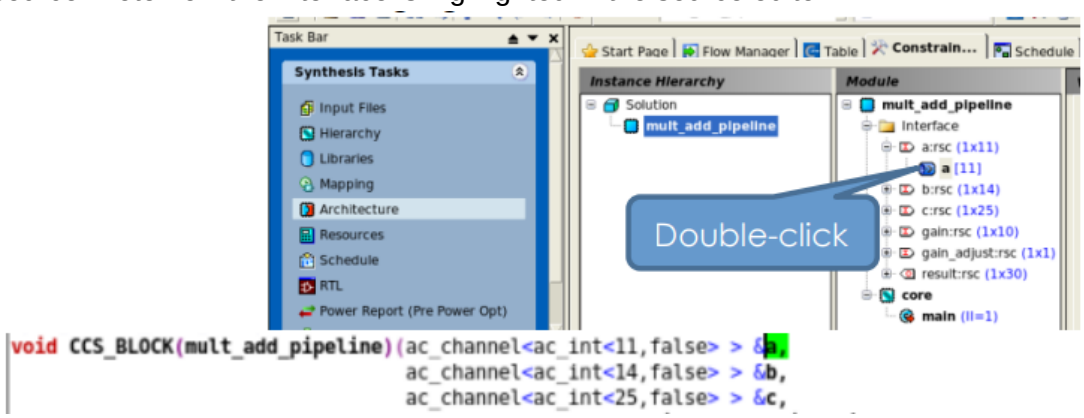
10. Click on Libraries in the Task Bar. These labs use the Nangate 45nm sample libraries and the OasysRTL flow. In a production design this is where you would select your technology and synthesis flow.



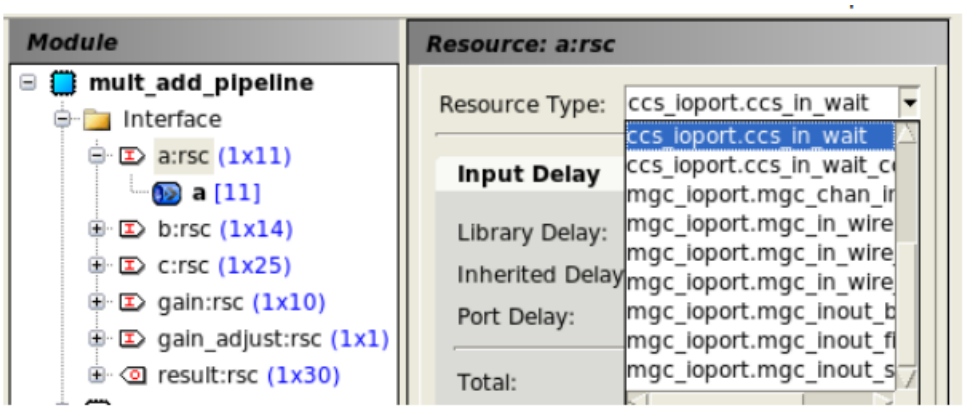
11. Click on Mapping in the Task Bar. This is where the clock frequency constraints are set as well as other constraints covered later. Select the “core” icon and note that the clock frequency is set to 900.9MHz.



12. Click on Architecture in the Task Bar. This opens the Architectural constraints dialog. This is where design constraints are applied.
    a. Expand the Interface folder and note the interfaces that have been created for the C++ design.
    b. Note that the bit-widths are shown along with the port direction.
    c. Expand the a:rsc interface and double-click on the “a” variable to go back to the source. Note how the interface is highlighted in the source editor

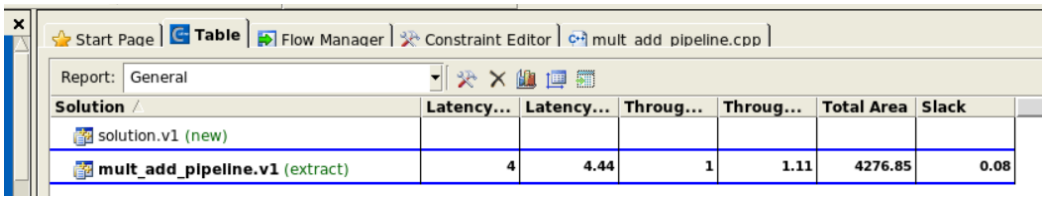


    d. Select a:rsc and note that it has been mapped to a ccs_in_wait interface which is the default for ac_channel interfaces. Other interfaces are selectable from the drop-down menu.



e. Click through the other resources under the interface folder. “a”, “b”, “c”, and “result” are all mapped to *_wait interfaces.

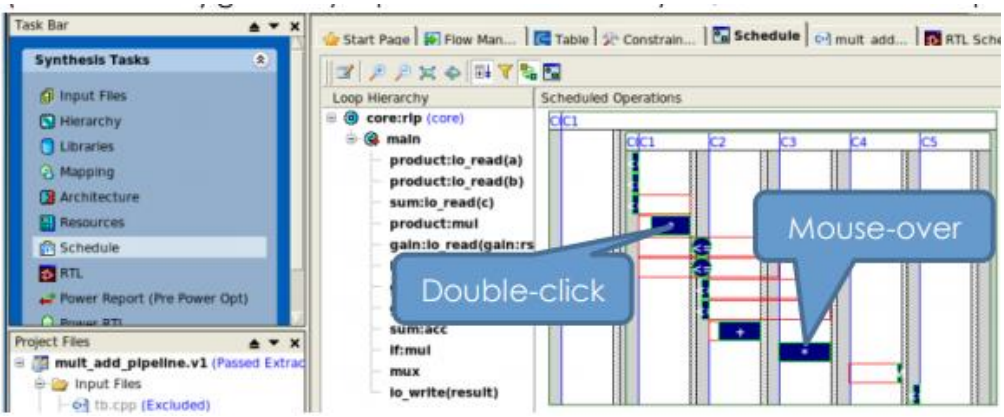
13. Click on the Table View Tab and note that Catapult reports a latency of 4 and a throughput of 1 for this design.



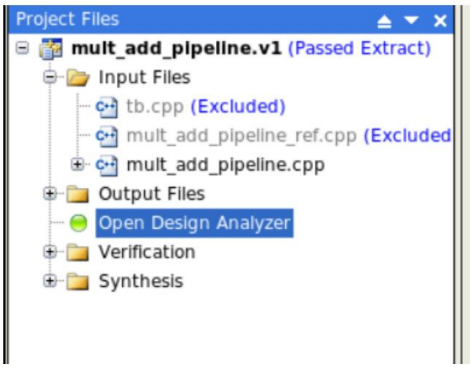
14. Click on Schedule in the Task Bar. This will display the Catapult Gantt Chart which provides a dataflow view of the schedule design.

a. Double-click on the operations in the Gantt chart to cross-probe back to the C++

b. Mouse over the operations to see the area and delay. Note that they are drawn to scale. The c-step (white column) generally represents one clock cycle, which in this example is 1.11ns



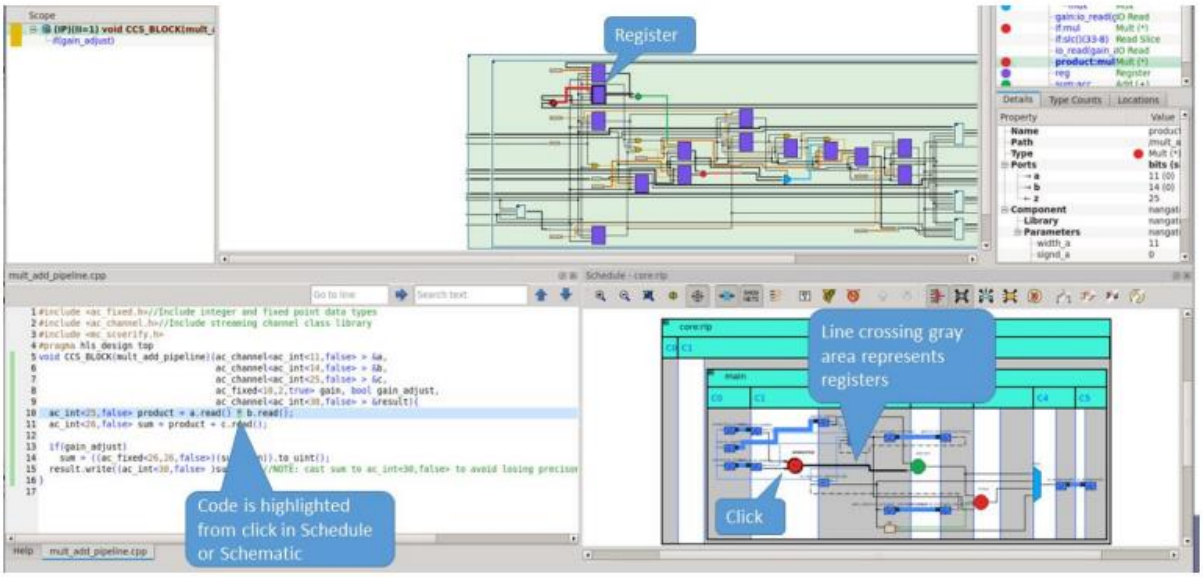
15. In the Project Files view double-click on “Open Design Analyzer”. This will bring up the Catapult Design Analyzer tool which allows designers to see exactly how the hardware is created from the C++ code and design constraints.



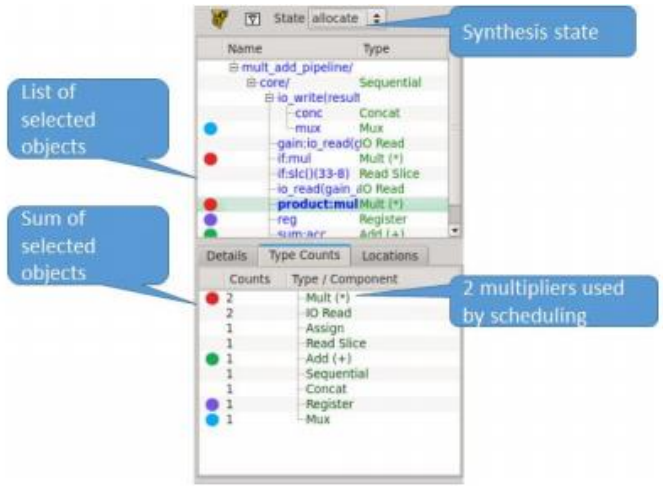
16. In Design Analyzer click on the first multiplier in the Schedule view. Note how the multiplier in the C++ code is highlighted. Note that the multiplier in the RTL schematic is also highlighted. Note also that the lines (data dependencies) that cross the c-step boundary (gray area) imply registers which can be seen in the RTL schematic.

17. Click through the other operations in the schedule view and note how they map back to the C++ and to the RTL schematic.

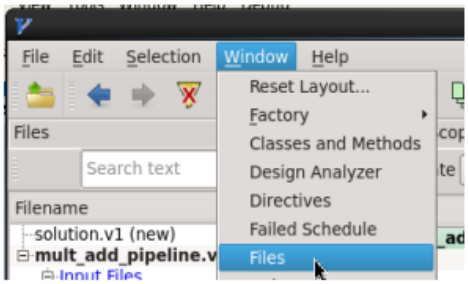
18. Click on the "blue" objects in the Schedule view. These are the IO. Note how they map back to the C++.



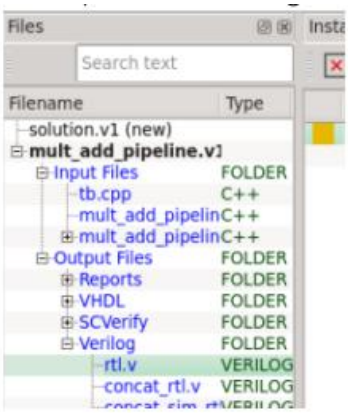
19. Click on the "Type Counts" Tab in the Design Analyzer Design Analyzer window. This gives a summary of all the objects for the current view and synthesis state.



20. Change the State of the Design Analyzer to Extract to see the hardware resources used for RTL generation. This includes registers and control logic added after scheduling.
21. Go to the Window menu and select Files.

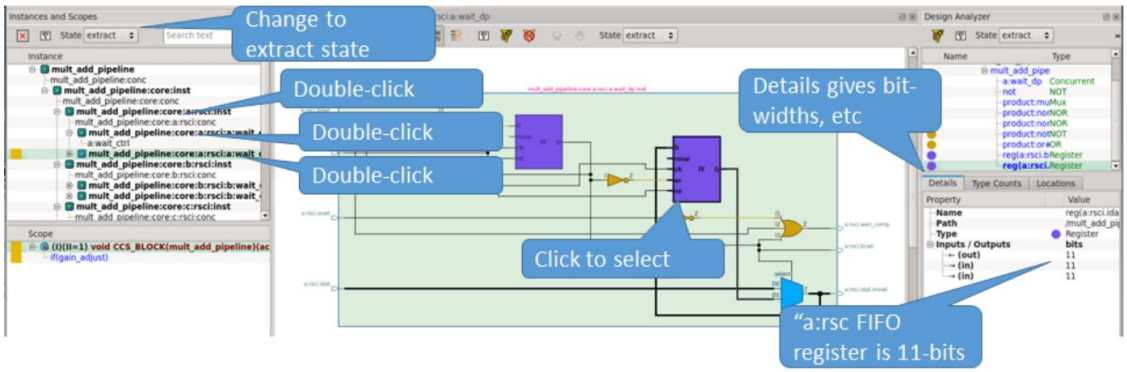


22. Double-click on rtl.v in the Files window to open the RTL design.



23. Click through the Schedule view and the RTL schematic and note how the RTL is highlighted. NOTE – not everything is cross-probable. There are some things in the schedule view that may not exist in the RTL.
24. Change the state of the Instances and Scopes view to extract. This will show all of the RTL instances added by synthesis such as IO, FSM, staller, etc.
    a. Double-click on the a:rsci:inst block which is the interface for the “a” ac_channel.

b. Double-click on the two sub-blocks under a:rsci:inst which are the FIFO and wait controller. Click on the register in the wait_dp block (1 element FIFO) and look at the details. This 11-bit register is the 1-element FIFO on the “a” input interface.



25. Close Design Analyzer

26. Close Catapult